Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Bu ders altı öğrenci sunumundan oluşur. Sunumlar dersin mikroservis kitabındaki bölümlere dayanır: süreçler arası iletişim (IPC), senkron uzak prosedür çağrısı (REST, gRPC, kısmi hata, servis keşfi), saga ile işlem koordinasyonu, iş mantığının düzenlenmesi (Transaction Script, Domain Model, DDD), domain event'lerin yayınlanması ve event sourcing ile sagaların birlikte kullanımı. Sunan kişiler aşağıda "bir öğrenci" diye anılır. Birinci sunum İngilizcedir ve altyazısının büyük kısmı bozuktur; bu kısım çoğunlukla slaytlardan aktarılmıştır.

Genel Konular

- Birinci sunum: mikroservis mimarisinde süreçler arası iletişime (IPC) genel bakış
  - Kısa tekrar: monolitik mimari küçük ekipler ve küçük projeler için basittir, hızlı geliştirme sağlar; ama bütün uygulama tek birim olarak ölçeklenir. Mikroservis mimarisinin asıl gücü servislerin bağımsız dağıtılabilen birimler olmasıdır.
  - Etkileşim stilleri (interaction styles): istemci ile servisin nasıl etkileştiğini teknolojiden bağımsız anlatma yolu. Önce bunu düşünmek, IPC ayrıntılarına takılmadan gereksinime odaklanmayı sağlar (ekrandan, 00:13:10).
    - İki boyut: bire bir (one-to-one, her isteği tek servis karşılar) ve bire çok (one-to-many, isteği birden çok servis karşılar); senkron (istemci zamanında cevap bekler, beklerken bloke olabilir) ve asenkron (istemci bloke olmaz, cevap hemen gelmeyebilir).
    - Tablo (ekrandan, 00:13:10):

	Bire bir	Bire çok
Senkron	Request/response	–
Asenkron	Asenkron request/response, tek yönlü bildirim (one-way notification)	Publish/subscribe, publish/async responses

    - Request/response: istemci servise istek yapar ve cevabı bekler. Publish/subscribe: istemci bir bildirim mesajı yayınlar, sıfır ya da daha fazla ilgili servis bunu tüketir.
  - API tanımlama
    - Büyük uygulamalar çok sayıda modülden oluşur; her modülün istemcilerinin kullanabileceği işlemleri tanımlayan bir arayüzü olmalıdır. İyi tasarlanmış arayüz işe yarar işlevi sunar, gerçekleştirmeyi gizler; gerçekleştirme istemcileri etkilemeden değişebilir (gevşek bağlılık).
    - Java'da arayüz bir dil yapısıyla (interface) tanımlanır ve gerçekleştirme sınıfı istemciden gizlidir.
    - API-first tasarım (ekrandan, 00:20:00): önce arayüz tanımı yazılır, sonra istemci geliştiricileriyle gözden geçirilir, birkaç tur yinelemeden sonra servis inşa edilir. Süreç tersine çevrilir: önce servis yazılıp sonra API düşünülmez.
    - API tanımının doğası seçilen IPC mekanizmasına bağlıdır: mesajlaşmada API mesaj kanalları, tipleri ve formatlarından; HTTP'de URL'ler, HTTP fiilleri ve istek/cevap formatlarından oluşur (ekrandan, 00:20:00).
  - API'lerin evrimi

        * Modern uygulamalar bakım için kapatılmaz (7/24 çalışır); bu yüzden eski ve yeni sürümün birlikte çalıştığı kademeli güncelleme (rolling update) yapılır.
        * Anlamsal sürümleme (semantic versioning), `MAJOR.MINOR.PATCH` (ekrandan, 00:22:40): MAJOR uyumsuz değişiklikte, MINOR geriye uyumlu iyileştirmede, PATCH geriye uyumlu hata düzeltmesinde artırılır.
        * REST API'de major sürüm URL yolunun ilk öğesi olarak kullanılabilir; mesajlaşma kullanan serviste sürüm numarası yayınlanan mesaja konabilir.
        * Genel kural: yalnız geriye uyumlu değişiklik yapmaya çalışmak. Sağlamlık ilkesi: "Yaptığın işte tutucu, başkalarından kabul ettiğinde hoşgörülü ol." Servis eksik istek alanlarına varsayılan değer verir, istemci fazladan gelen cevap alanlarını yok sayar. Geriye uyumlu değişiklik örneği: isteğe ek alan eklemek.
        * Major değişiklik örneği: istemci sürümü HTTP başlığında ister (ekrandan, 00:24:40):
          ```
          GET /orders/xyz HTTP/1.1
          Accept: application/vnd.example.resource+json; version=1
          ...
          ```
    - Mesaj formatları
        * IPC'nin özü mesaj alışverişidir; mesaj formatı seçimi IPC verimliliğini, API'nin kullanılabilirliğini ve evrimini etkiler.
        * Metin formatları (JSON, XML): insan tarafından okunur ve kendini tanımlar. JSON adlandırılmış özelliklerin, XML adlandırılmış öğelerin ve değerlerin koleksiyonudur. Tüketici ihtiyacı olanı alır, gerisini yok sayar.
        * İkili formatlar: Protocol Buffers (Google) ve Avro (Apache) (ekrandan, 00:27:40). Protocol Buffers etiketli alanlar (tagged fields) kullanır; Avro tüketicisi mesajı yorumlamak için şemayı bilmelidir, bu yüzden API evrimi Protocol Buffers ile daha kolaydır. İkisi de mesaj yapısı için tipli bir IDL sunar; derleyici serileştirme kodunu üretir; bu da API-first yaklaşımını zorunlu kılar. İstemci statik tipli bir dilde yazılırsa derleyici API'nin doğru kullanıldığını denetler.
- İkinci sunum: senkron uzak prosedür çağrısı (Remote Procedure Invocation, RPI)
    - RPI: istemci, bir vekil (proxy) arayüzü üzerinden servise istek gönderir ve cevabı alır. Uzaktaki fonksiyon ve kaynak çağrıları bu kapsama girer. Yöntemler: REST, gRPC, webhook, WebSocket.
    - REST: Roy Fielding'in tanımladığı, ölçeklenebilir bileşen iletişimi için HTTP tabanlı bir stil. Kaynaklar (resource) temsilleri (representation) üzerinden, insan okuyabilir JSON ya da XML ile alınıp verilir.
        * Özellikler: net istemci-sunucu ayrımı (gevşek bağlılık), insan okuyabilir metin, HTTP/1.1 ve HTTP/2 desteği, GET/POST gibi temel komutlar, durumsuzluk (stateless: her istek önceki isteklerden bağımsız, ilk kez görülüyormuş gibi cevaplanır), önbelleğe alınabilirlik (cacheable), tek biçimli arayüz.
        * REST olgunluk modeli: HTTP tabanlı her API RESTful değildir. Yalnız POST kullanılırsa düzey 0-1'de kalınır; diğer HTTP fiilleri de kullanılırsa düzey 2'ye, HATEOAS ilkesi uygulanırsa düzey 3'e çıkılır.
        * REST API'nin belirtimi: REST'in başta IDL'si yoktu; topluluk bunun değerini yeniden keşfetti; en yaygın REST IDL'i Swagger projesinden doğan OpenAPI Specification'dır (ekrandan, 00:35:50).
        * Sorunlar: tek istekte birden çok kaynağa erişmek zordur (çözüm önerileri: GraphQL, Netflix Falcor); bazı işlemler HTTP fiilleriyle ifade edilemez.
        * Artılar: basit ve tanıdık (HTTP fiillerini kullandığı için web geliştiricileri kolay anlar); güvenlik duvarına takılmaz. Eksiler: yalnız request/response stilini destekler; yukarıdaki iki sorun.
    - gRPC: Google'ın geliştirdiği, HTTP/2 kullanan, Protocol Buffers ile ikili mesajlaşan, dil ve platform bağımsız bir yöntem (Java, C#, Node.js, Go vb.) (ekrandan, 00:40:00).
        * Artılar: zengin güncelleme işlemleri olan API'yi tasarlamak kolaydır; büyük mesajlarda verimli ve kompakt IPC; çift yönlü akış (bidirectional streaming) hem RPI hem mesajlaşma stilini mümkün kılar; çok sayıda dil arasında birlikte çalışabilirlik.
        * Eksiler: JavaScript istemcileri için daha fazla iş; eski güvenlik duvarları HTTP/2'yi desteklemeyebilir; ikili format insan tarafından okunmaz. REST gibi senkron olduğu için o da kısmi hata (partial failure) sorununu yaşar.
    - Kısmi hata ve Circuit Breaker

        * Cevap vermeyen bir servise giden istekler birikirse çağıran tarafın kaynakları (ör. thread) tükenir ve uygulama çalışamaz hale gelir. Kitaptaki örnek: mobil uygulama → API gateway → OrderServiceProxy → cevap vermeyen Order Service (ekrandan, 00:41:20).
        * Önlemler: ağ zaman aşımı (her isteğe süre sınırı), istek sayısı sınırı (bir istemcinin bekleyen istek sayısını sınırlamak), Circuit Breaker.
        * Circuit Breaker deseni (ekrandan, 00:41:20): ardışık hata sayısı bir eşiği aşınca RPI vekili bir zaman aşımı süresi boyunca çağrıları hemen reddeder.
        * Çöken servisten toparlanma: ya hata döndürmek ya da önbellekteki eski cevap gibi bir yedek (fallback) değer döndürmek.
    - Servis keşfi (service discovery)
        * İstek yapmak için servisin ağdaki konumu (IP adresi ve port) bilinmelidir; günümüzde adresler dinamik olduğu için servis keşfi gerekir.
        * İki yaklaşım: servis kayıt defteri (service registry) kullanmak (servisler kendini kaydettirir, istemci kayıtlı örneklerden birine yük dengelemeyle gider) ve dağıtım altyapısının (Docker, Kubernetes gibi platformlar, sanal IP) sağladığı keşif.
        * Platform keşfinin artısı: bütün dil ve çerçevelerle çalışır. Eksisi: ör. Kubernetes yalnız kendi üzerinde dağıtılmış servisleri keşfeder.
- Üçüncü sunum: saga ile işlem koordinasyonu
    - Mikroservis: bir uygulamayı bağımsız çalışan, küçük, birbirinden izole servisler olarak tasarlama yöntemi. Artıları: dayanıklılık, esneklik, ölçeklenebilirlik, modülerlik. Zorluklarından biri: servislere yayılan dağıtık işlemlerde veri tutarsızlığı.
    - Para transferi örneği: hesap mikroservisi müşteri hesabından parayı düşer, transfer mikroservisi parayı alıcıya yatırır. Monolitte ikisi aynı veri tabanında tek bir atomik transaction olurdu. Mikroservislerde her servisin kendi veri tabanı vardır; para düşüldükten sonra transfer servisi hata verirse tutarsızlık oluşur. Saga ile telafi işlemi (parayı müşteriye geri yatırma) eklenir.
    - ACID (ekrandan, 00:50:00): bütünlük (atomicity), tutarlılık (consistency), yalıtım (isolation), dayanıklılık (durability).
    - Saga: birden çok mikroservise yayılan işlemleri yönetme deseni. Her adım bir öncekinin başarısıyla tetiklenir; hata olursa önceki adımlar geri alınır (rollback/telafi).
    - Koreografi tabanlı saga: merkezi yönetici yoktur; her servis işini bitirince event yayınlar, sonraki servis o event'i tüketir. E-ticaret örneği (ekrandan, 00:54:40):
      ```
      Order Service    -> ORDER_CREATED_EVENT (sipariş Pending)
      Payment Service  dinler, ödemeyi alır    -> BILLED_ORDER_EVENT
      Stock Service    dinler, stoğu günceller -> ORDER_PREPARED_EVENT
      Delivery Service dinler, kargolar        -> ORDER_DELIVERED_EVENT
      Order Service    dinler, siparişi tamamlandıya çeker
      ```
        * Hata akışı: Stock Service `PRODUCT_OUT_OF_STOCK` event'i yayınlarsa Payment Service ödemeyi geri alır, Order Service siparişi başarısız (fail) durumuna çeker.
        * Artılar: basit, yeni servis eklemek kolay, dağıtık ve bağımsız çalışma. Eksiler: karmaşık senaryolarda izleme ve hata yönetimi zorlaşır, servisler arası event trafiği artar, test için bütün servislerin çalışır olması gerekir.
    - Orkestrasyon tabanlı saga: tek sorumluluğu katılımcılara ne yapacaklarını söylemek olan bir orkestratör sınıfı vardır; komut ve geri dönüş (reply) etkileşimi kullanılır. Örnek (ekrandan, 01:00:00):
      ```
      Order Service    -> sipariş Pending, Orchestrator'a ORDER_CREATED
      Orchestrator     -> EXECUTE_PAYMENT -> Payment Service -> Payment Executed Reply
      Orchestrator     -> UPDATE_STOCK    -> Stock Service   -> Order Prepared Reply
      Orchestrator     -> ORDER_DELIVER   -> Shipment/Delivery Service -> Order Delivered Re
      (cevaplar Order Saga Reply Channel üzerinden orkestratöre döner)
      ```
        * Her adım için saga üzerinde durum (state) tutmak, sürecin hangi adımda yanlış gittiğini görmeyi kolaylaştırır. Hata durumunda orkestratör rollback'i koordine eder.
        * Artılar: merkezi kontrol, uygulaması ve test edilmesi daha kolay, yeni adım eklenince karmaşıklık koreografiye göre daha az artar, birbirini bekleyen işlemlerde uygundur. Eksiler: ek orkestratör sınıfı ve altyapı karmaşıklığı (maliyet).
- Dördüncü sunum: iş mantığını düzenleme desenleri
    - Altıgen mimari (ekrandan, 01:05:30): iş mantığı merkezdedir, kullanıcı arayüzü ve veri tabanı gibi çevre sistemlerden bağımsızdır; her parça ayrı test edilebilir.

    - Gelen (inbound) adaptörler: REST API (`POST /orders`, `GET /order/Id`), Order command handlers (sipariş servisi isteklerini iş mantığına iletir).
    - Giden (outbound) adaptörler: Domain event publisher adapter (iş mantığının ürettiği önemli olayları diğer servislere iletir), Database adapter (veri tabanı işlemlerini iş mantığından soyutlar).
  - İki yaklaşım: nesneye yönelik (gerçek dünya varlıklarını sınıflarla temsil etme) ve prosedürel (işlem adımları ve metotlar). Seçim uygulamanın büyüklüğüne ve karmaşıklığına bağlıdır.
  - Transaction Script deseni (ekrandan, 01:08:20): her istek ayrı bir prosedürle ele alınır. Davranış sınıfı (`OrderService`: `createOrder()`, `reviseOrder()`, `cancelOrder()`), veri erişim nesnesi (`OrderDao`: `save(Order)`, `findOrderById()`) ve yalnız durum tutan veri sınıfı (`Order`: `orderId`, `orderLineItems`).
    - Artılar: basit, hızlı, her işlem bir yönteme karşılık gelir. Eksiler: iş mantığı büyüyünce tek servis sınıfında karmaşıklaşır; kalıtım gibi nesneye yönelik olanaklardan yararlanılamaz.
    - Ne zaman: iş mantığı basit ve işlem odaklıysa, nesneye yönelik model gereksizse (ekrandan, 01:10:00).
  - Domain Model deseni: hem durum hem davranış içeren, gerçek dünya kavramlarını temsil eden küçük sınıflar. `OrderService` yalnız yönlendirir; işlemler `Order` gibi domain sınıflarındadır; `DeliveryInformation` gibi değer sınıfları ve `OrderRepository` vardır.
    - Artılar: anlaşılması, güncellenmesi ve test edilmesi kolay; Strategy ve Template Method gibi tasarım desenleriyle genişletilebilir. Eksi: model büyüyünce sınıflar birbirine bağımlı hale gelebilir.
  - Domain-Driven Design (DDD): alan uzmanlarıyla yazılımcılar arasında ortak dil kurmayı amaçlayan metodoloji. Her servisin kendi domain modeli vardır; sınırlı bağlam (bounded context) içinde kendi kurallarıyla çalışır.
    - Stratejik desenler: alanı alt alanlara (subdomain) böler.
    - Taktiksel desenler: domain modeldeki sınıfların rollerini düzenler: entity (benzersiz kimlik, ör. `Order`), value object (kimliksiz değer koleksiyonu, ör. teslimat bilgisi), servis, factory (ör. statik oluşturma metodu), repository (veri tabanı işlemlerini soyutlar).
    - Artılar: modülerlik, anlaşılabilirlik, test edilebilirlik, genişletilebilirlik. Aggregate kavramı bir sonraki sunuma bırakılmıştır.
- Beşinci sunum: domain event'lerin yayınlanması
  - Event: dikkate değer bir durumun meydana gelmesi; DDD'de iş alanında meydana gelen önemli bir durumu temsil eder. Örnekler: Sipariş Oluşturuldu (Order Created), Sipariş İptal Edildi (Order Cancelled), Sipariş Gönderildi (Order Shipped) (ekrandan, 01:20:00). Adları geçmiş zaman fiiliyle verilir.
  - Domain event'ler aggregate'lerle ilişkilidir; aggregate önemli bir durum değişikliğine uğradığında domain event yayınlanır.
  - Kullanım nedenleri: kullanıcı davranış analizi (sepete ekleme olayı), sistem doğruluğunun izlenmesi ve geri bildirim (ATM'de para çekme), başka iş süreçlerini tetikleme (ödeme onaylanınca sipariş hazırlığının başlaması), sistem içi bileşenleri tetikleme (ayar değişince dosyaya yazma).
  - Event sınıfı: anlamlı özellikler ve meta veriler (event ID, zaman damgası, isteğe bağlı kullanıcı kimliği). Meta veri ya event nesnesinin içinde ya da genel (generic) bir zarf (`DomainEventEnvelope`) içinde tutulur.
  - Event zenginleştirme (event enrichment): tüketicinin ihtiyaç duyduğu bilgiyi event'e koymak. Alternatif, tüketicinin kaynağa sorgu atmasıdır (performans maliyeti). Zenginleştirmenin eksisi: gereksinim değiştikçe event sınıfı da değişir (bakım maliyeti).
  - Domain event'leri belirleme: gereksinimlerden yola çıkma ("X olduğunda Y'yi yap"; ör. "Yemek kuryeye teslim edildiğinde tüketiciye yola çıktı bildiriminin gönderilmesi" → `DagitimaCikti`) (ekrandan, 01:30:00) ve event storming.
  - Event storming: alan uzmanları büyük bir tahta ve renkli yapışkan notlarla birkaç saatte kaba bir model çıkarır. Adımlar: event'ler üzerine beyin fırtınası (turuncu notlar, zaman çizelgesine), event tetikleyicilerini (komutlar mavi, aktörler, dış sistemler) belirleme, aggregate'leri (sarı) belirleme. Örnek: otopark sistemi; `Create new Ticket` komutu → `TicketCreated` event'i; Ticket machine, Parking, Payment aggregate'leri (ekrandan, 01:34:45).

  - Event üretme ve yayınlama: aggregate mesajlaşma API'sini doğrudan çağırmaz (altyapı ile iş mantığı karışır). Aggregate durum değişince event'leri bir liste olarak döndürür; servis bunları `DomainEventPublisher` ile yayınlar.
  - Güvenilir yayın (ekrandan, 01:38:30): event'ler yerel veri tabanı işleminin parçası olarak gönderilmelidir. Transactional messaging, aggregate güncellemesini ve event yayınını tek ACID işleminde birleştirir; event'ler OUTBOX tablosuna yazılır, işlem bitince mesaj aracısına gönderilir. Eventuate Tram çerçevesi bunu uygular.
  - Event tüketimi: `DomainEventDispatcher` event'leri uygun handler'lara yönlendirir (ör. Kitchen Service'in restoran menüsünü güncel tutması).
  - Örnek sistem: kütüphanede kitap ödünç alma (`KitapOduncAlindiEvent`, `Kitap` aggregate'i, `KitapServisi`) (ekrandan, 01:41:00).
- Altıncı sunum: event sourcing ve sagaların birlikte kullanımı
  - Saga türlerinin karşılaştırması: koreografide merkezi yönetici ve tek hata noktası (single point of failure) yoktur, basit akışlar için iyidir; ama servis eklendikçe takip zorlaşır, döngüsel bağımlılık riski vardır, test için bütün servislerin ayakta olması gerekir. Orkestrasyonda döngüsel bağımlılık riski kalkar, karmaşık akışlar için idealdir; ama orkestratör ayrıntılı tasarlanmalıdır ve orkestratör çökerse sistem durur (tek hata noktası).
  - Event sourcing: veriyi doğrudan değiştirmek yerine veri üzerindeki bütün değişiklikleri event olarak bir event store'da saklamak.
  - Event sourcing ve koreografi sagaları birlikte iyi çalışır (ekrandan, 01:50:00): mesajlaşma tabanlı IPC, idempotent handler'larla mesaj tekrarının önlenmesi, aggregate'in atomik güncellenmesi. Zorluk: event'ler iki amaca hizmet eder; veri değişmese bile (ör. iş kuralı ihlali) sagaya yanıt event'i gerekir.
  - Orkestrasyon sagası oluşturma: `createOrder()` gibi servis metotları hem aggregate oluşturur hem saga orkestratörü başlatır; ilki yapılırsa ikincisinin de yapılacağı garanti edilmelidir. İlişkisel event store'da tek transaction yeter; NoSQL event store'da servis, domain event'e karşılık saga orkestratörünü oluşturan event handler kullanır; event ID ile tekrarlanan mesajlar elenir.
  - Event sourcing tabanlı saga katılımcısı: iki sorun idempotent mesaj işleme (mesaj kimliğini kaydedip tekrarları atma) ve atomik yanıt gönderme. Çözüm (ekrandan, 01:55:50): event handler aggregate'i güncellerken gerçek event'lerle birlikte sahte bir `SagaReplyRequested` event'i kaydeder; bu event'in handler'ı yanıt mesajını oluşturup orkestratörün yanıt kanalına yazar.
  - Event sourcing ile saga orkestratörü: orkestratörün durumu `SagaOrchestratorCreated` ve `SagaOrchestratorUpdated` event'lerinden yeniden kurulur; gönderilecek her komut için hedef kanal ve komut içeren bir `SagaCommandEvent` kaydedilir (en az bir kez gönderim); yanıtlar mesaj kimliği saklanarak tek kez işlenir.

Hocanın Özellikle Vurguladığı Kısımlar

- Bu derste hoca yalnız sunumları yönetti; akademik bir vurgu ya da ek açıklama yapmadı.
  - Hoca, saga sunumunun IPC sunumundan sonra gelmesinin konu ve mantık akışı açısından daha iyi olduğunu belirtti.
- Sunumlarda öne çıkan, kitabın da altını çizdiği noktalar:
  - REST ve gRPC senkron mekanizmalardır, bu yüzden ikisi de kısmi hata sorununu yaşar; Circuit Breaker, zaman aşımı ve istek sınırı bu yüzden gerekir.
  - API'ler kontrollü evrilmeli; mümkün olduğunca yalnız geriye uyumlu değişiklik yapılmalıdır.
  - Aggregate mesajlaşma API'sini doğrudan çağırmamalı; event'i servis yayınlamalı ve yayın veri tabanı işlemiyle aynı transaction'da (outbox) yapılmalıdır.

Kısa Tekrar Notları

- Etkileşim stilleri: bire bir/bire çok × senkron/asenkron. Request/response, asenkron request/response, tek yönlü bildirim, publish/subscribe, publish/async responses.
- API-first: önce arayüz tanımı, istemcilerle gözden geçirme, sonra servis.
- Semantic versioning: MAJOR uyumsuz, MINOR geriye uyumlu iyileştirme, PATCH geriye uyumlu hata düzeltmesi. Sağlamlık ilkesi: tutucu gönder, hoşgörülü kabul et.

- Mesaj formatları: metin (JSON, XML; okunur, kendini tanımlar) ve ikili (Protocol Buffers tagged fields; Avro şema gerektirir).
- REST: kaynak + temsil, stateless, cacheable, HTTP fiilleri; olgunluk düzeyleri 0-3 (düzey 3 = HATEOAS); IDL olarak OpenAPI.
- gRPC: HTTP/2 + Protocol Buffers, çok dilli, çift yönlü akış; JavaScript istemcisi ve eski güvenlik duvarı sorunları.
- Kısmi hata: zaman aşımı, istek sınırı, Circuit Breaker; toparlanmada hata ya da yedek (önbellek) değer.
- Servis keşfi: servis kayıt defteri ya da platform (Kubernetes, Docker) keşfi.
- Saga: dağıtık işlemi yerel işlemler dizisine böler, hata olursa telafi eder. Koreografi = event'ler, merkezi yönetici yok. Orkestrasyon = orkestratör komut gönderir, yanıt alır.
- Transaction Script = davranış sınıfı + DAO + veri sınıfı, basit iş mantığı için. Domain Model = durum + davranış sınıfları, karmaşık iş mantığı için.
- DDD: bounded context; stratejik (subdomain) ve taktiksel (entity, value object, service, factory, repository) desenler.
- Domain event: geçmiş zaman adı, meta veri, zenginleştirme; event storming ile belirleme; outbox ile güvenilir yayın.
- Event sourcing: durum yerine değişiklik event'leri saklanır; sagalarla birlikte idempotent işleme ve atomik yanıt sorunları çözülmelidir (`SagaReplyRequested`, `SagaCommandEvent`).

Detaylı Açıklamalar

- Etkileşim stilleri ve IPC seçimi
  - Kitap, bir servisin API'sini tasarlamadan önce etkileşim stilini düşünmeyi önerir. Bire bir ve bire çok ayrımı bir isteği kaç servisin karşıladığını, senkron ve asenkron ayrımı ise istemcinin cevabı bekleyip beklemediğini anlatır. Senkron bire çok etkileşim tabloda boştur.
  - API'nin ne içereceği IPC mekanizmasına bağlıdır. Mesajlaşmada kanallar, mesaj tipleri ve formatları; HTTP'de URL'ler, fiiller ve istek/cevap formatları API'yi oluşturur.
- API evrimi
  - Mikroservis tabanlı bir uygulamada bir servisin istemcileri başka servisler ve başka ekipler olabilir. Ayrıca uygulama bakım için kapatılmadığından eski ve yeni sürüm birlikte çalışır (rolling update). Bu yüzden sürümleme ve geriye uyumluluk kritik hale gelir.
  - Geriye uyumlu değişiklikte eski istemciler yeni servisle çalışmaya devam eder: servis eksik alanlara varsayılan değer verir, istemci fazla alanları yok sayar. Uyumsuz (major) değişiklikte istemci istediği sürümü URL yolunda ya da HTTP `Accept` başlığında (`version=1`) belirtir ve servis bu başlığa göre ilgili sürümü sunacak bir mekanizmaya sahip olmalıdır.
- Mesaj formatları
  - Metin formatları okunabilir ve kendini tanımlar; tüketici yalnız ilgilendiği alanları alır. İkili formatlarda Protocol Buffers etiketli alanlar kullanır, Avro tüketicisi şemayı bilmek zorundadır; bu yüzden API evrimi Protocol Buffers ile daha kolaydır. İki formatta da IDL'den kod üretildiği için API önce tanımlanmak zorundadır.
- REST'in sınırları ve gRPC
  - REST, kaynak odaklı ve HTTP fiillerine bağlı olduğu için "birden çok kaynağı tek istekte getirme" ve "fiillerle ifade edilemeyen işlemler" gibi durumlarda zorlanır. GraphQL ve Netflix Falcor ilk soruna, gRPC ise ikinci soruna yöneliktir. gRPC'de API, Protocol Buffers IDL'iyle tanımlanır ve zengin işlem kümeleri kolayca tasarlanır.
- Kısmi hata
  - Kitaptaki şekilde mobil uygulama `POST /orders` isteğini API gateway'e gönderir, gateway içindeki `OrderServiceProxy` isteği Order Service'e iletir. Order Service cevap vermezse naif bir vekil süresiz bekler; her bekleyen istek bir thread tutar, sonunda gateway'in kaynakları tükenir ve bütün API kullanılamaz hale gelir. Çözüm iki parçalıdır: RPI vekillerini cevap vermeyen servislere karşı tasarlamak (zaman aşımı, istek sınırı, Circuit Breaker) ve çöken servisten nasıl toparlanılacağına karar vermek (hata döndürmek ya da yedek değer).
- Saga koordinasyonu
  - Para transferi örneği sagaya neden ihtiyaç olduğunu gösterir: ayrı veri tabanları tek bir ACID transaction'a sokulamaz. Saga, her biri kendi veri tabanında yerel transac-

tion olan adımlardan oluşur; bir adım başarısız olursa önceki adımların etkisi telafi işlemleriyle geri alınır.
  - Koreografide akış event'lerin kim tarafından dinlendiğine dağılmıştır; bu basit akışta zarif, büyük akışta izlenmesi zordur. Orkestrasyonda akış orkestratörde toplanır; adım eklemek ve test etmek kolaylaşır, ama orkestratör ek bir bileşendir.
- İş mantığı desenleri
  - Transaction Script'te iş mantığı servis sınıfının metotlarındadır, veri sınıfları davranışsızdır. Domain Model'de davranış domain sınıflarına dağılır, servis sınıfı yalnız yönlendirir. DDD, domain modelin büyüyünce birbirine bağımlı hale gelmesini sınırlı bağlamlar ve aggregate'lerle önler.
- Domain event yayınlama
  - Aggregate durumunu değiştiren metot, ürettiği event'leri bir liste olarak döndürür; servis önce aggregate'i kaydeder, sonra event'leri yayınlar. Event yayını ile veri tabanı güncellemesi ayrı yapılırsa biri olup diğeri olmayabilir; outbox tablosu ikisini aynı yerel transaction'a sokar.
  - Kütüphane örneğinde event sınıfı (ekrandan, 01:41:00):

```
public class KitapOduncAlindiEvent implements DomainEvent {
    private final String kitapId;
    private final String kullaniciId;
    private final ZonedDateTime alisTarihi;

    public KitapOduncAlindiEvent(String kitapId, String kullaniciId, ZonedDateTime ali
        this.kitapId = kitapId;
        this.kullaniciId = kullaniciId;
        this.alisTarihi = alisTarihi;
    }

    public String getKitapId() { return kitapId; }
    public String getKullaniciId() { return kullaniciId; }
    public ZonedDateTime getAlisTarihi() { return alisTarihi; }
}
```

    * Kod parçası derlenmedi (derste gösterilen slayttan aktarıldı).
- Event sourcing ve sagalar
  - Event sourcing'te event zaten mesaj olarak yayınlandığı için koreografi sagaları doğal olarak uygulanır. Zorluk, saga katılımcısının yanıtı atomik göndermesi ve mesajları tek kez işlemesidir; kitap bunu `SagaReplyRequested` sahte event'i ve mesaj kimliklerinin saklanmasıyla çözer. Orkestratörün kendisi de event sourcing ile saklanabilir: durumu oluşturma/güncelleme event'lerinden kurulur, gönderilecek komutlar `SagaCommandEvent` olarak kaydedilir.
- Ders sonrası notlar
  - Sunan öğrenci REST olgunluk modelini REST'i tanımlayan kişiye (Roy Fielding) bağladı; bu model büyük olasılıkla Leonard Richardson'a aittir (Richardson Maturity Model).
  - ACID slaytında dayanıklılık (durability) "hata olursa verilerin önceki haline döndürülebilmesi" diye tanımlandı (ekrandan, 00:50:00). Doğrusu: dayanıklılık, onaylanmış (commit edilmiş) bir transaction'ın sonucunun sistem çökse bile kalıcı olmasıdır; hata durumunda geri alma bütünlüğün (atomicity) konusudur.
  - Circuit Breaker sözlü anlatımda başarılı/başarısız istek oranına göre "o istemciden gelen istekleri reddetme" diye anlatıldı. Slayttaki tanıma göre (ekrandan, 00:41:20) ardışık hata sayısı eşiği aşınca vekil, cevap vermeyen servise yapılan çağrıları bir zaman aşımı süresince hemen reddeder.
  - Altıncı sunumda idempotence bir yerde "mesajın en az bir kez işleneceğinin garantisi" diye geçti. Doğrusu: en az bir kez (at-least-once) teslim mesajlaşma altyapısının garantisidir; idempotence, aynı mesaj birden çok kez işlense de sonucun değişmemesidir.
  - İkinci sunum girişte "asenkron mesajlaşma" dedi; anlatılan RPI (REST, gRPC) ise senkron iletişimdir (ekrandan, 00:40:00: "like REST, it's a synchronous communication mechanism").
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli

çalışabilirsiniz.